



CURSO BÁSICO DE PROGRAMACIÓN EN JAVA

MANEJO DE ARCHIVOS

Tutor/es: Brandon Jiménez, Christopher Benavides, Jhon Laverde,
Dayana Jácome

1

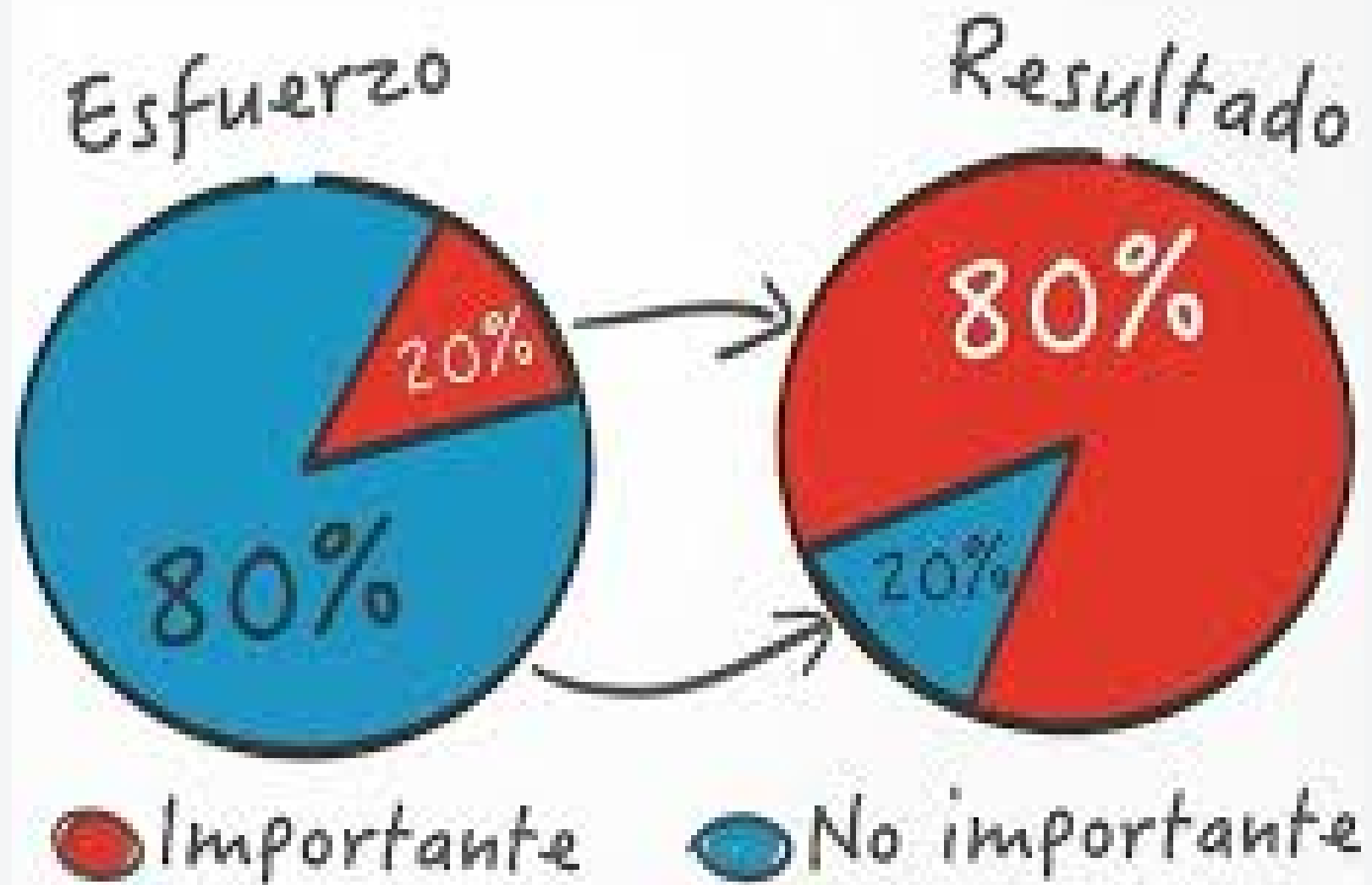


AGENDA

Contenidos del Proyecto

01. Introducción	5
02. P00	6 - 10
03. Reutilizar y Automatizar	11
04. I/O de datos	12-14
05. Tarea	15





REGLA DE PARETO

80%/20%

El 80% de los resultados es a causa del 20% del código. Los errores que nuestro programa da al ejecutarse, generalmente es un error en la lógica del código o falta de priorizar importancia en funciones críticas o más importantes. Priorizar el 20% de las características que aportan el 80% del valor al usuario, lo que permite lanzar un Producto Mínimo Viable (MVP) más rápido.

4



INTRODUCCIÓN

Archivos

En la Programación es importante manejar archivos txt, pdf, csv, doc, sql, etc., con estos, podemos crear nuestra información y almacenarlo para luego mostrar o procesarlo como mejor nos parezca, incluso en un Data Warehouse.



5

FEEDBACK/RETROALIMENTACIÓN

Clase, Objeto, Constructores

Es un diseño o plano. Permite crear un objeto en base a ese diseño.
Constructor, permite darle valores iniciales.

Atributos y métodos

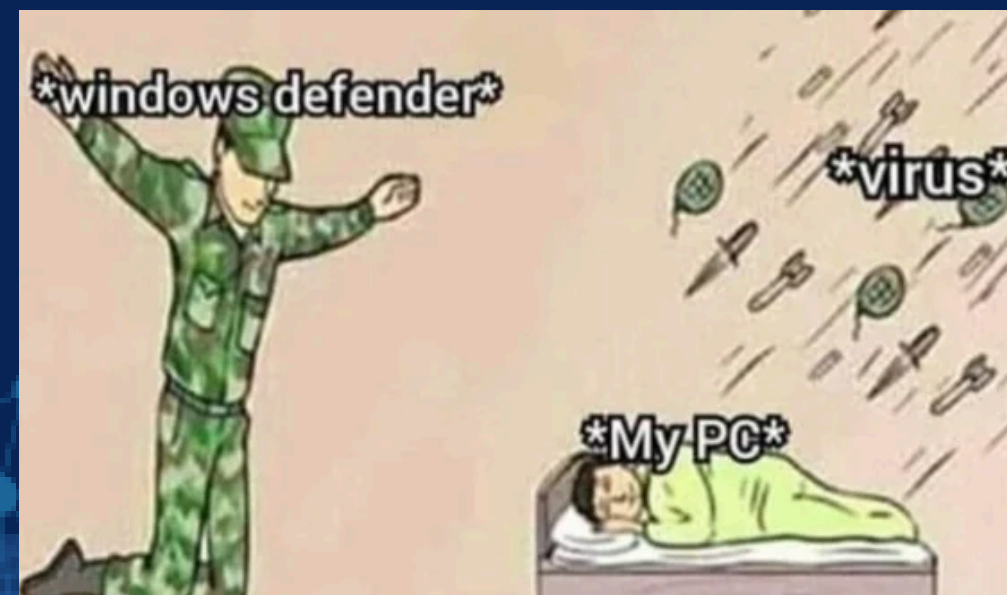
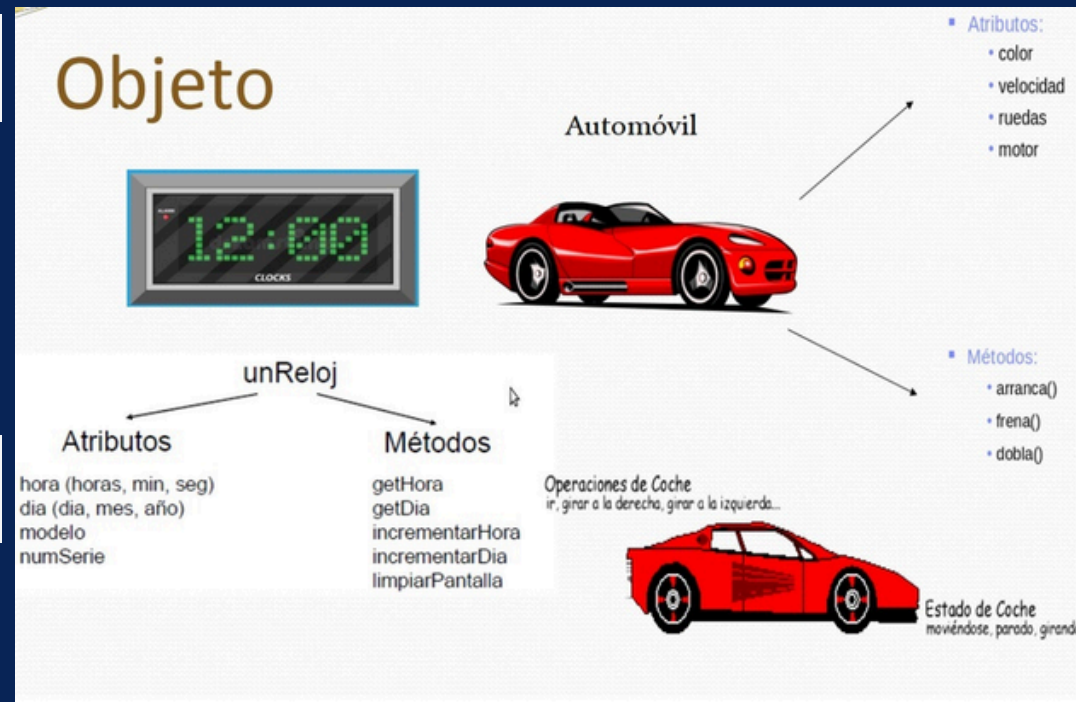
Definen características de una Clase y el comportamiento que presentan.

Encapsulamiento

Permite proteger los atributos y solo sus funciones puedan usar los usuarios.

Excepciones

Permite controlar errores de diferentes tipos.



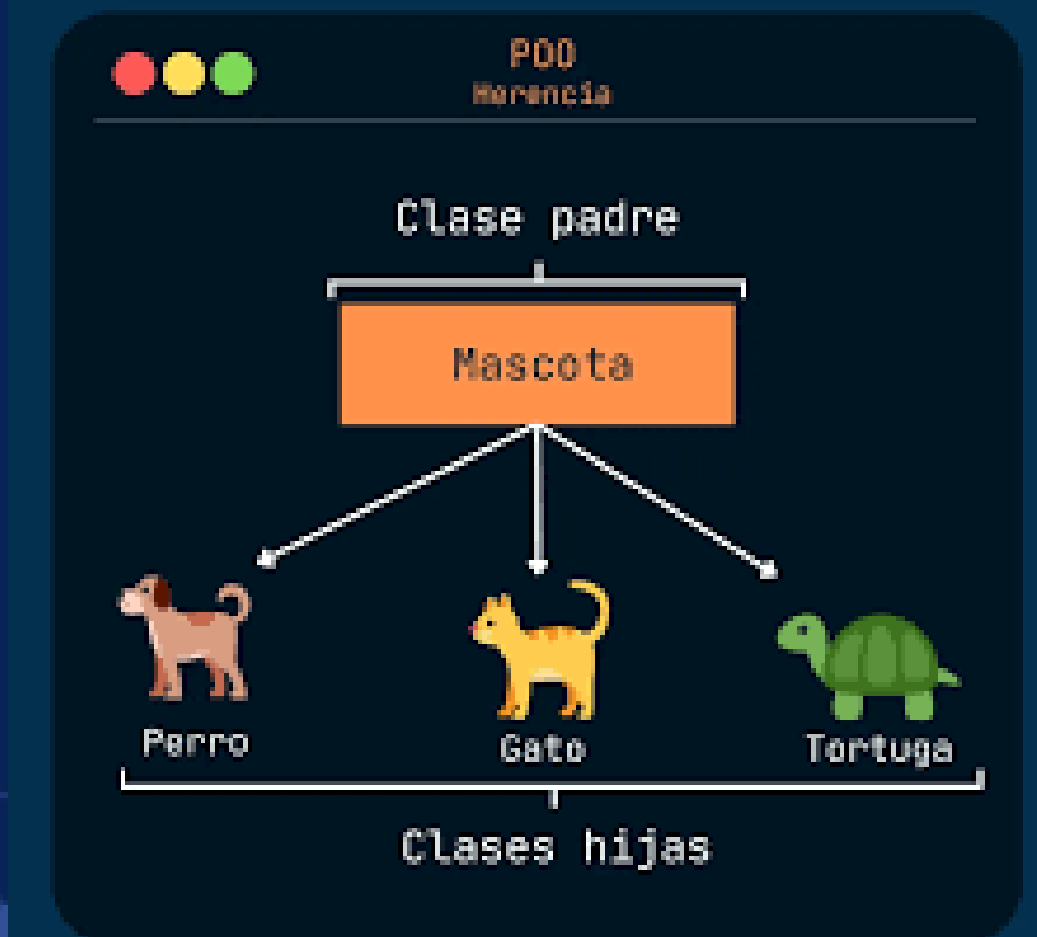


¿QUE ÉS HERENCIA?

Explicación

La herencia, permite heredar atributos y métodos de otra Clase, esto los distinguimos con el termino de Padre (super clase) e Hija (subclase).

La Clase Hija hereda las características del Padre, pero pueden tener otros atributos y compartamiento adicionales.

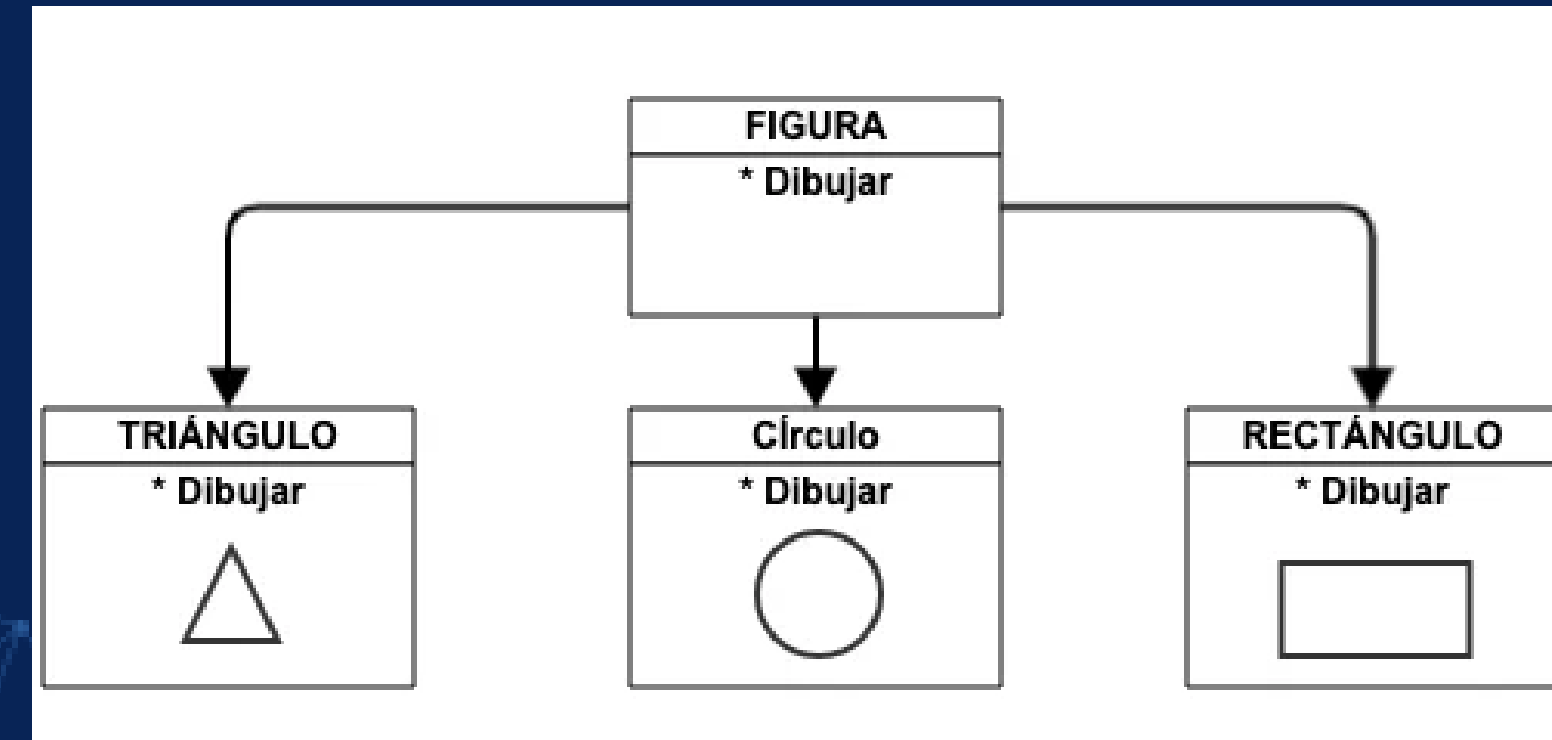


¿QUE ÉS ABSTRACCIÓN?

Explicación

La Abstracción representa un objeto o concepto de forma simple. Deja los detalles irrelevantes para un problema. Existen dos tipos: Clases Abstractas e Interfaces. Las Abstractas no pueden inicializarse sino que heredan sus funciones a otras clases para extenderlas. Las interfaces se centra en los métodos o comportamientos por ser implementados.

Nota: Al extender una Clase o implementarla, sus métodos pueden variar, aplicando el principio de Polimorfismo. Un mismo método, diferentes comportamientos.





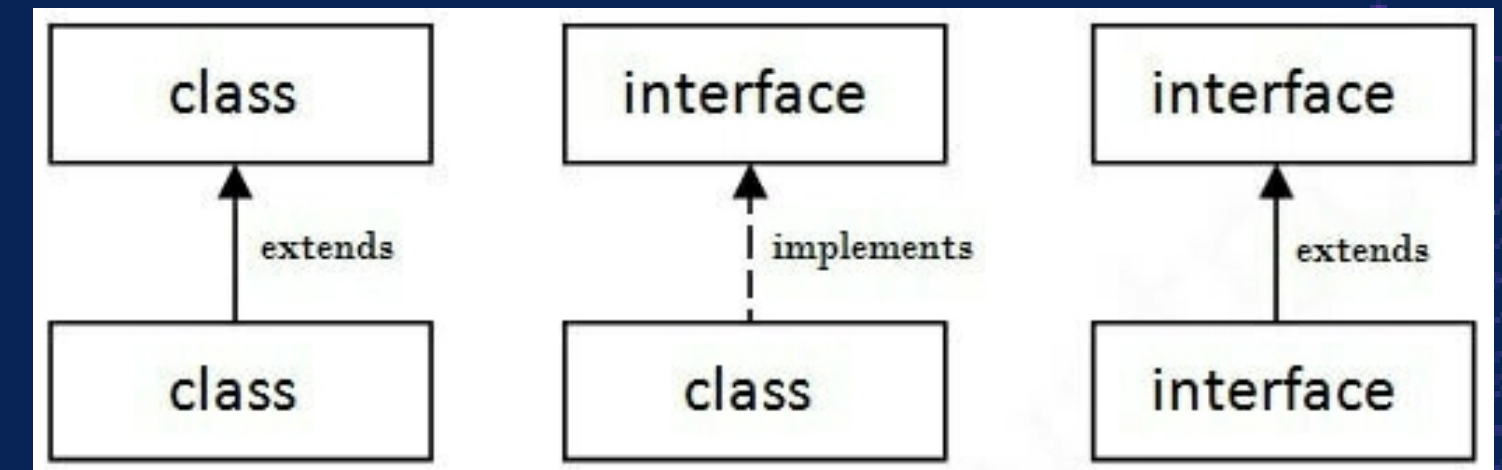
¿QUE ÉS UNA INTERFAZ?

Explicación

Una Interfaz genera un contrato o firma de atributos y métodos. Es similar a una Clase Padre/Abstracta. La diferencia es que pueden hacer multi implementación de diferentes interfaces en una clase y generalmente se centran en una base de comportamiento.

Se sugiere evitar atributos/características en interfaces.

Nota: Las Interfaces ayudan a cumplir el **Principio de Sustitución de Liskov (LSP)**: Las subclases deben ser sustituibles por sus clases base sin alterar el funcionamiento del programa. (**Método SOLID**)





PILARES DE POO

Principios POO



ABSTRACCIÓN

Permite crear un modelo Padre que define que hace y sus Hijos explicaran como hacerlo.



HERENCIA

Permite heredar las clases Padre en las clases Hijas. Teniendo su atributos y métodos. Pueden Sobreescribirse.



ENCAPSULAMIENTO

Permite proteger los atributos o métodos ante modificaciones externas no autorizadas. Usando public, private o protected. Si no se define una variable se considera de acceso default o package-private



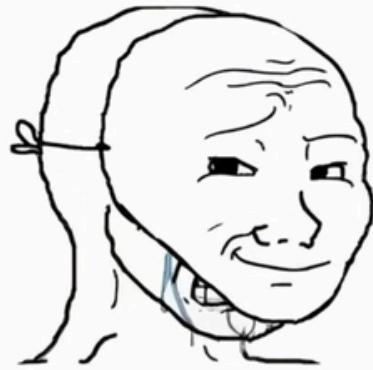
POLIMORFISMO

El Polimorfismo permite que muchas Clases Hijos puedan sobreescribir las funciones del Padre y responderlas de diferente manera.

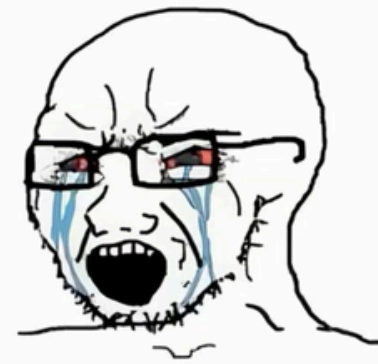
10

REUTILIZAR Y AUTOMATIZAR

DISEÑADORES



Mira, nuestras ideas se parecen.



¡No! Robaste mi idea.

PROGRAMADORES



Colega, robé tu código.



No era mío.



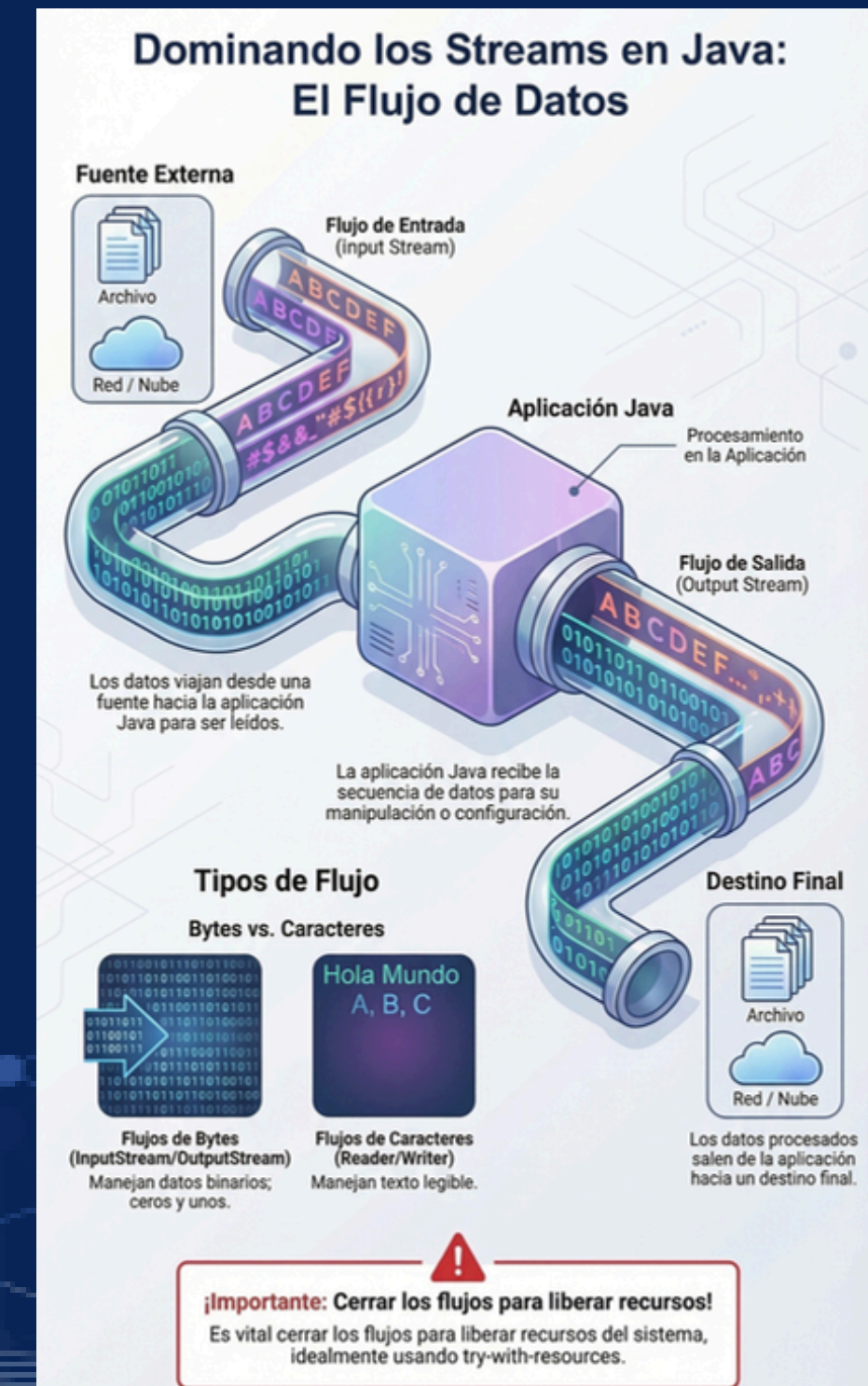
¿QUÉ ES UN STREAM?

Explicación

Son una secuencia de datos que se procesan de manera continua. Los flujos son fundamentales para la programación, ya que permiten leer y escribir los tipos de datos de manera eficiente.

Flujos de entrada (Input Streams) → leen archivos, conexión de red, etc.

Flujos de Salida (Output Streams) → Permite la escritura de datos.



ENTRADA DE DATOS

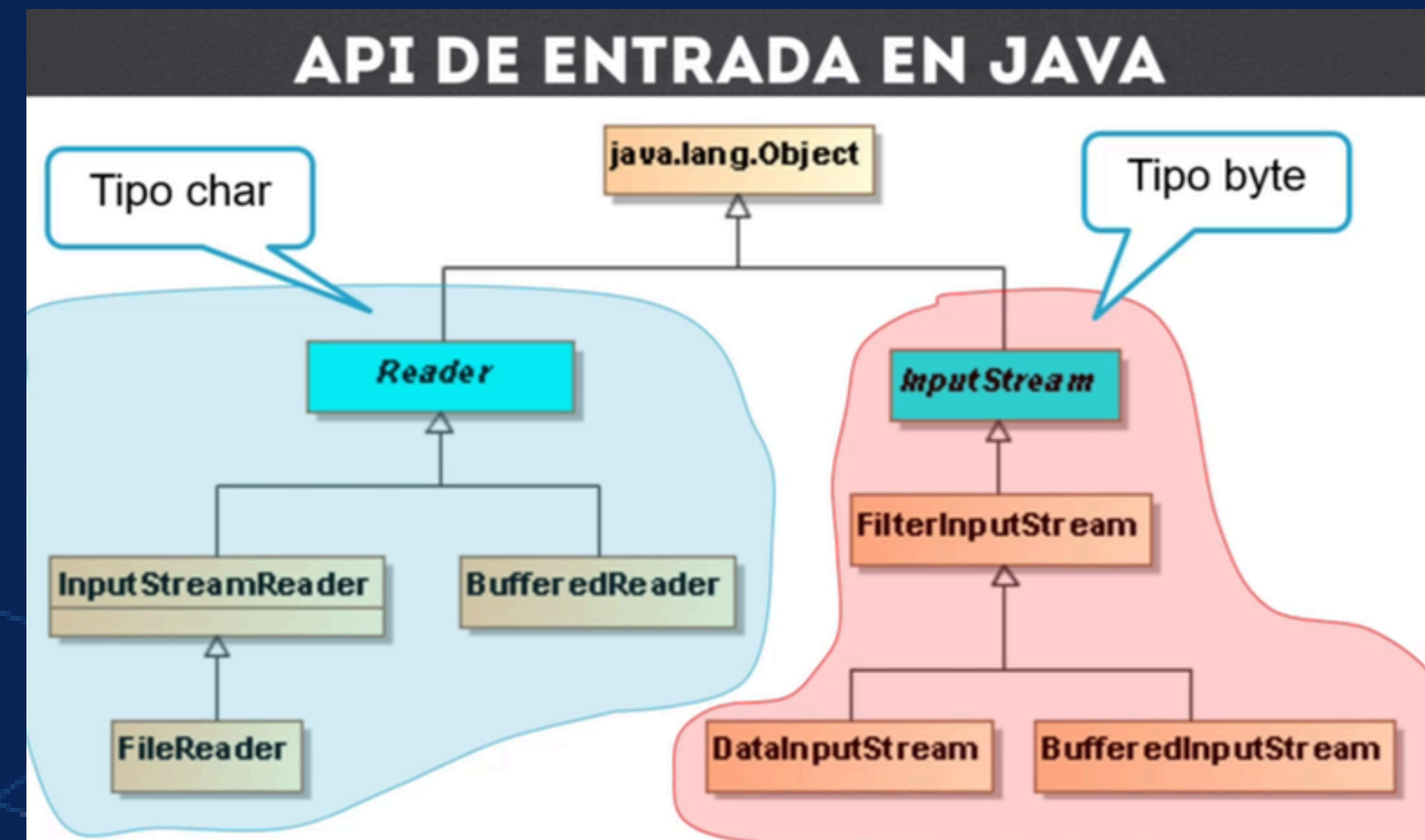
Explicación

Las entradas de datos pueden ser a nivel de texto y bits.

InputStream procesa datos de entrada de tipo archivo Byte (8 bits). Donde manejan documentos Doc, Pdf, Excel, Exe, Img, , db, etc.

Reader procesa datos de entrada a nivel de texto 16 bits (2 Bytes - Unicode) como Txt, CSV, log, md, py, java, php, etc. Soporte a UTF-8.

Nota: Los archivos que se manejen a nivel de Bits usa **InputStream**. Y los que son legibles, generalmente se ocupan **Reader**.



SALIDA DE DATOS

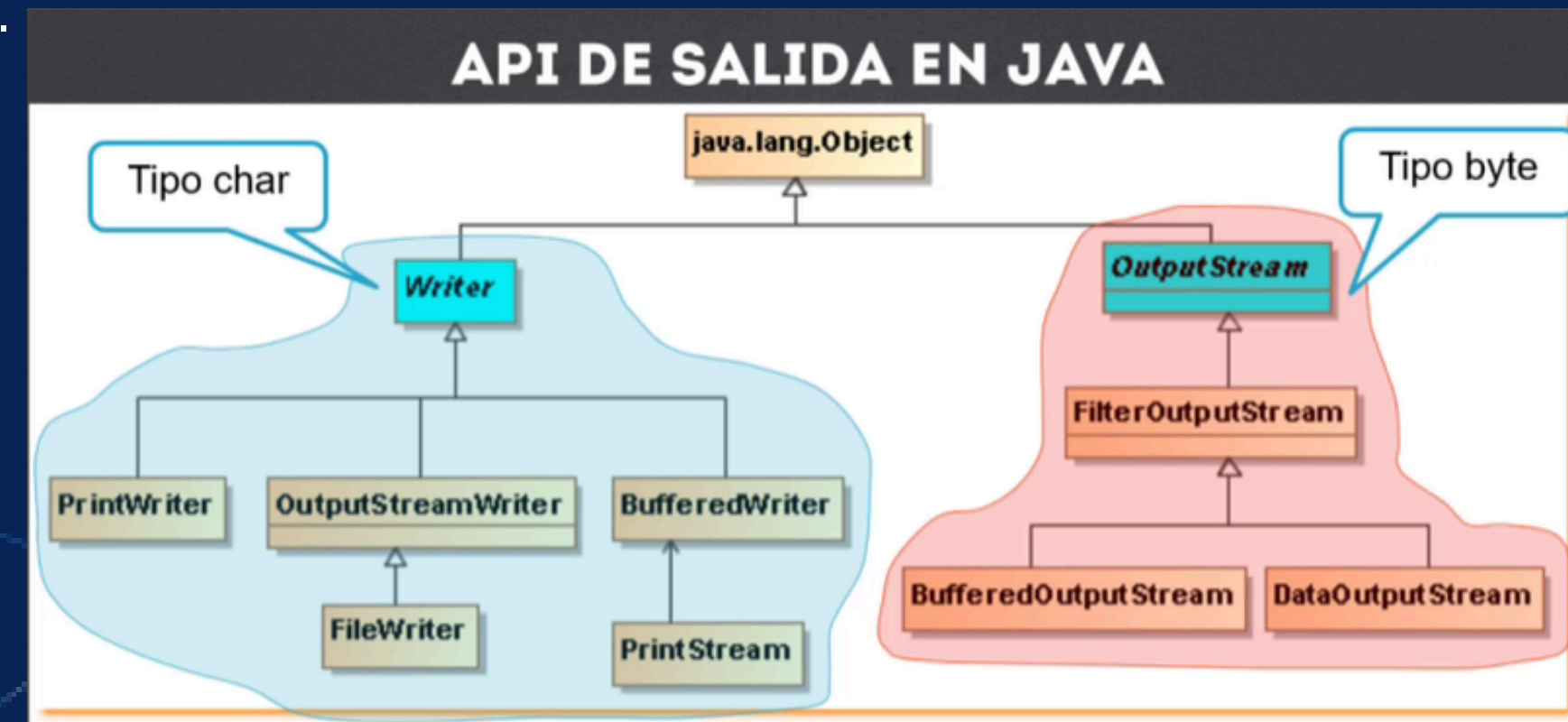
Explicación

Las Salidas de datos pueden ser también a nivel de texto y bits.

OutputStream escribe datos en archivos de tipo Binarios o Bytes. Para guardar cambios, es necesario cerrar el archivo.

Writer escribe datos en archivos de tipo Texto. Para guardar cambios, es necesario cerrar el archivo.

Nota: Los archivos es el primer paso, para llegar a Base de Datos o entenderlo. Cada conexión que abras requiere ser cerrado, para guardar información, ahorrar recursos del equipo, etc.



System.in → Permite capturar la entrada de dato

InputStreamReader → Accede a la consola estandar y lee 1 caracter a la vez.

BufferedReader → Permite capturar más de una línea de caracteres ingresados.

Try - catch → Al leer o escribir archivos, puede surgir problemas, se sugiere usar.

readLine → Permite leer la primera línea del mensaje. Devuelve en String. Y hace un salto de línea.

While → Permite crear un bucle para lectura de mensaje

IOException → Permite captura errores de entrada y salida.

printStackTrace → imprime un rastreo de pila en el flujo de salida de error que es el valor del campo System.err.

LECTURA ENTRADA ESTANDAR EN JAVA

Uso del API de Entrada de Datos en Java:

```
import java.io.*;

public class EntradaDatos {

    public static void main(String args[]) {
        String captura;
        InputStreamReader input = new InputStreamReader(System.in);
        BufferedReader bInput = new BufferedReader(input);
        try {
            System.out.println("Introduce un dato:");
            captura = bInput.readLine();
            while (captura != null) {
                System.out.println("Dato introducido:" + captura);
                captura = bInput.readLine();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```


File → Representación abstracta de rutas de archivos y directorios

FileWriter → Orientado a caracteres, escribe texto en archivos

PrintWriter → Permite crear un archivo o sobrescribe si existe en disco duro.

Close → Permite cerrar el archivo. Y los datos se crean en el documento.

Try - catch → Al leer o escribir archivos, puede surgir problemas, se sugiere usar.

IOException → Permite captura errores de entrada y salida.

printStackTrace → imprime un rastreo de pila en el flujo de salida de error que es el valor del campo System.err.

CREACION DE ARCHIVOS EN JAVA

Creación de archivos en Java:

```
import java.io.*;

public class ManejoArchivos {

    public static void main(String[] args) {
        File archivo = new File("c:\\pruebaJava\\pruebaJava.txt");
        try {
            PrintWriter salida = new PrintWriter(new FileWriter(archivo));
            salida.close();
        } catch (IOException ex) {
            ex.printStackTrace();
        }
        System.out.println("El archivo se ha creado correctamente");
    }
}
```



File → Representación abstracta de rutas de archivos y directorios.

FileReader → Permite la lectura de caracteres.

readLine → Permite leer líneas de texto y un salto de línea para leer la siguiente.

Close → Permite cerrar el archivo. Y los datos se crean en el documento.

Try - catch → Al leer o escribir archivos, puede surgir problemas, se sugiere usar.

IOException → Permite capturar errores de entrada y salida.

FileNotFoundException → Permite capturar errores de Tipo Archivo o File.

LECTURA DE UN ARCHIVO EN JAVA

Lectura de un archivo en Java:

```
import java.io.*;

public class LecturaArchivos {

    public static void main(String args[]) {
        File archivo = new File("c:\\pruebaJava\\pruebaJava.txt");
        try {
            BufferedReader entrada = new BufferedReader(new FileReader(archivo));
            String lectura;
            lectura = entrada.readLine();
            while (lectura != null) {
                System.out.println("Lectura: " + lectura);
                lectura = entrada.readLine();
            }
            entrada.close();
        } catch (FileNotFoundException ex) {
            ex.printStackTrace();
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

File → Representación abstracta de rutas de archivos y directorios.

FileWriter → Orientado a caracteres, escribe texto en archivos

PrintWriter → Permite crear un archivo o sobrescribe si existe en disco duro.

println → Permite ingresar texto en una línea.

Close → Permite cerrar el archivo. Y los datos se crean en el documento.

Try - catch → Al leer o escribir archivos, puede surgir problemas, se sugiere usar.

IOException → Permite captura errores de entrada y salida.

printStackTrace → imprime un rastreo de pila en el flujo de salida de error que es el valor del campo System.err.

Anexando información a un archivo:

```
package manejoarchivos;

import java.io.*;

public class AnexarInformacionArchivo {

    public static void main(String args[]) {
        File archivo = new File("c:\\pruebaJava\\pruebaJava.txt");
        try {
            PrintWriter salida = new PrintWriter(new FileWriter(archivo, true));
            String contenido = "Anexando informacion al archivo";
            salida.println(contenido);
            salida.println();
            salida.println("Fin de anexar");
            salida.close();
        } catch (IOException ex) {
            ex.printStackTrace();
        }
        System.out.println("Se ha anexado la informacion correctamente");
    }
}
```




TAREA

Una vez revisado y entendiendo archivos.
Se les propone el siguiente ejercicio

Crear un archivo llamado estudiantes.txt con una función crear.

Crear una función que permita registrar estudiantes.

Crear una función que permita mostrar los estudiantes.

No olvide el control de excepciones.

Estudiantes		
nombre	edad	nota
Luis	20	8
Maria	22	9
Carlos	19	7

```
nombre, edad, nota
Luis, 20, 8
María, 22, 9
Carlos, 19, 7|
```



GRACIAS

